

JSP View Layer의 진화: EL & JSTL 완벽 가이드

더 이상 JSP에 자바 코드를 섞어 쓰지 마세요.
퍼블리셔와 개발자 모두를 위한 클린 코드 아키텍처.

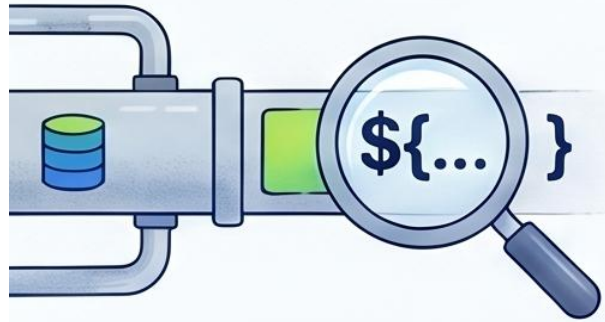
```
<% for(int i=0; i<list.size(); i++) { %>
    <% for(int i=0; i<list.size(); i++) { %>
        <%= list.get(i).getName() %>
        <%= list.get(i).getName() %>
        <%= list.get(i).getName() %>
    } %>
}%>
```

```
<ul>
  <c:forEach items='${list}' var='item'>
    <li>${item.name}</li>
  </c:forEach>
</ul>
```


JSP 개발의 핵심: EL과 JSTL로 자바 코드 최소화하기

JSP 페이지 내에서 자바 코드를 최소화하는 것은 유지보수와 협업의 핵심입니다. 표현 언어(EL)는 데이터를 화면에 쉽게 출력하도록 돕고, JSTL(표준 태그 라이브러리)은 자바의 제어문을 HTML 태그 형태로 대체하여 깔끔한 View를 완성합니다.

EL (Expression Language) - 데이터 출력의 단순화



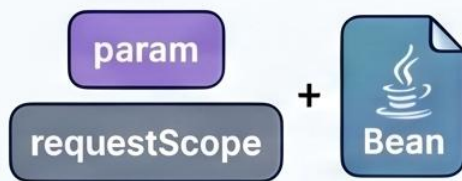
'\${데이터}' - 직관적인 데이터 출력

request나 session에 저장된 데이터를 자바 코드 없이 브라우저에 즉시 출력합니다.



유연한 연산 기능 지원

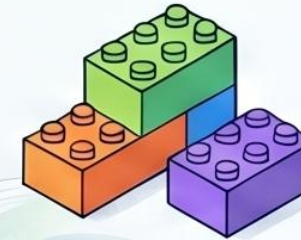
산술, 비교, 논리 연산뿐만 아니라 null을 0으로 처리하는 등 관대한 연산을 수행합니다.



내장 객체 및 빈(Beans) 접근

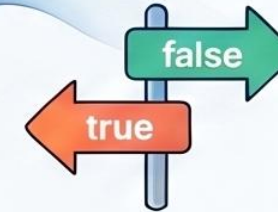
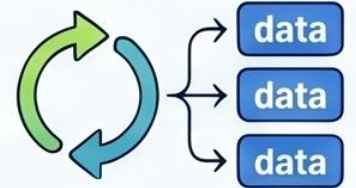
param, requestScope 등을 통해 파라미터나 객체의 Getter 메서드에 쉽게 접근합니다.

JSTL (Standard Tag Library) - 태그형 제어문



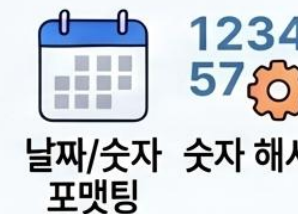
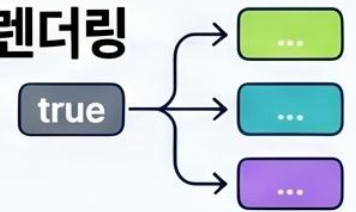
<c:forEach> - 반복문의 혁신

리스트나 배열 데이터를 HTML 태그 내에서 자바 루프 없이 반복 출력합니다.



<c:if> & <c:choose> - 조건부 렌더링

특정 조건에 따라 화면 요소를 표시하거나, 다중 조건문을 태그 형식으로 관리합니다.



'fmt' & 'fn' 라이브러리 활용

날짜/숫자 포매팅(fmt)과 문자열 처리(fn) 기능을 제공하여 데이터 가공을 돕습니다.



비교: 기존 자바 코드 vs EL/JSTL 대체 방식

기존 자바 코드 (Scriptlet)

데이터 출력: `<%= request.getAttribute("name") %>`
반복문: `<% for(SawonVO vo : list) { %>`
조건문: `<% if(score >= 90) { %>`

EL / JSTL 대체 방식

데이터 출력: `${name}`
반복문: `<c:forEach var="vo" items="${list}">`
조건문: `<c:if test="${score >= 90}">`

JSP의 진화: EL & JSTL로 완성하는 Scriptless JSP

JSP 페이지에서 자바 코드를 배제하고 EL과 JSTL을 사용하여 가독성과 협업 효율을 높이는 핵심 방법론

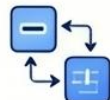
EL (Expression Language) - 데이터 접근의 간소화



자바 코드 없이 데이터 즉시 출력

`$(name)` 형식을 사용하여 request나 session에 저장된 객체를 Getter 메서드 호출 없이 간편하게 출력합니다.

`<% out.print(request.getAttribute("user")); %>` ❌ → `$(user)` ✅



똑똑한 산술 및 논리 연산자

null을 0으로 처리하는 산술 연산과 and, or, empty 등 직관적인 키워드를 통한 조건 판별이 가능합니다.



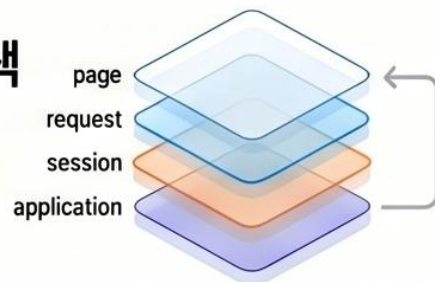
`null + 5 == 5`

`empty & and`



내장 객체를 통한 스코프 탐색

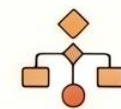
requestScope, sessionScope 등 내장 객체를 제공하며 영역 지정 생략 시 request를 우선 탐색합니다.



```
<%
if (...) {
  if (...) == 5 {
    out.println(...);
  }
}
for (for (i.t, i++) {
  out.println(...);
  out.println(...);
}
%>
```

```
${
  ${...}
  ${...}>
  <c:...>
  <:...>
}
```

JSTL (Standard Tag Library) - 태그 기반 제어 구조



<c:if>와 <c:choose>의 조건 분기

단일 조건은 <c:if>, 다중 조건이나 else 처리가 필요할 때는 <c:choose> <c:otherwise>를 사용합니다.

```
<c:if test="...">
  ...
</c:if>

<c:choose>
  <c:when test="...">
    ...
  <c:otherwise> ...
</c:choose>
```

구분	<c:if>	<c:choose>
지바 대응	if 문	switch, if-else if 문
Else 처리	지원 않 함 (주황 연산사 사용)	<c:otherwise>로 지원
주요 용도	단순 조건 확인	티입 분기 및 기종값 설정

조건문 처리를 위한
JSTL 태그 선택 가이드



<c:forEach>를 이용한 반복 처리

배열이나 리스트를 순회하며, varStatus의 index를 활용해 순번 표시나 다른 데이터 참조를 수행합니다.

```
<c:forEach item="..." var="item">
  item 1 : varStatus
  item 2 : 1
  item 3 : 2
  ... : 3
</c:forEach>
```



fmt 태그로 데이터 포맷팅

날짜(yyyy-MM-dd)나 숫자(화메 기호, 천 단위 콜마)를 특정한 자바 클래스 없이 태그 하나로 변환합니다.

<fmt:formatNumber>	<fmt:formatData>
1234567	₩1,234,567
20240101	2024-01-01

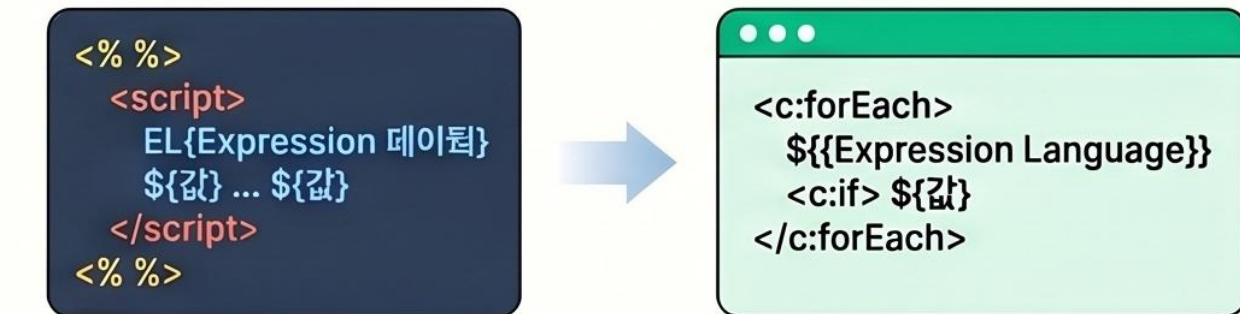
비즈니스 로직과 화면 디자인을 명확히 분리하여 유지보수성과 보안성을 극대화합니다.

JSP MVC 웹 개발 핵심 가이드: EL, JSTL & Oracle SQL

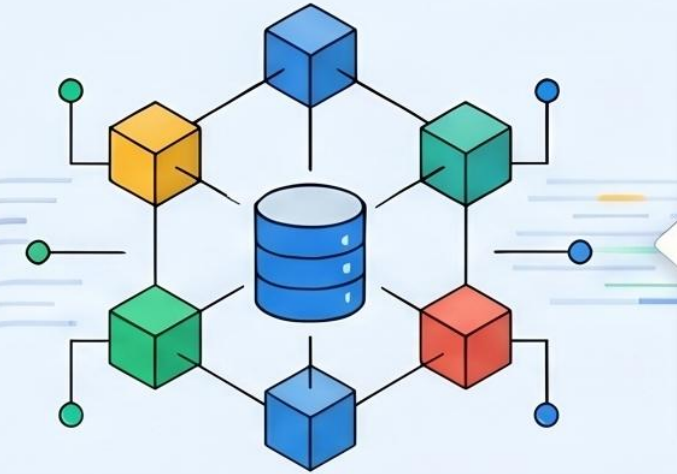
화면(View)의 효율화: EL & JSTL



실무 핵심 규칙: JSP 내 Java 코드 최소화



데이터(Model) 관리의 핵심: Oracle SQL



5 실무 필수 SQL 5대 키워드
데이터 관리에 반드시 필요한 키워드를 완벽히 숙달해야 함.

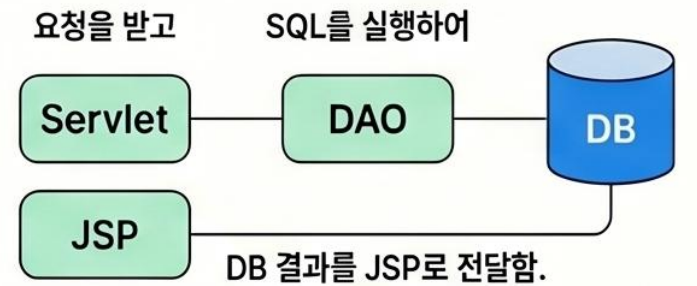


게시판 페이징과 시퀀스



rownum을 활용한 페이징 처리와 SEQUENCE를 이용한 번호 자동 증가가 실무 핵심임.

MVC 데이터 처리 흐름

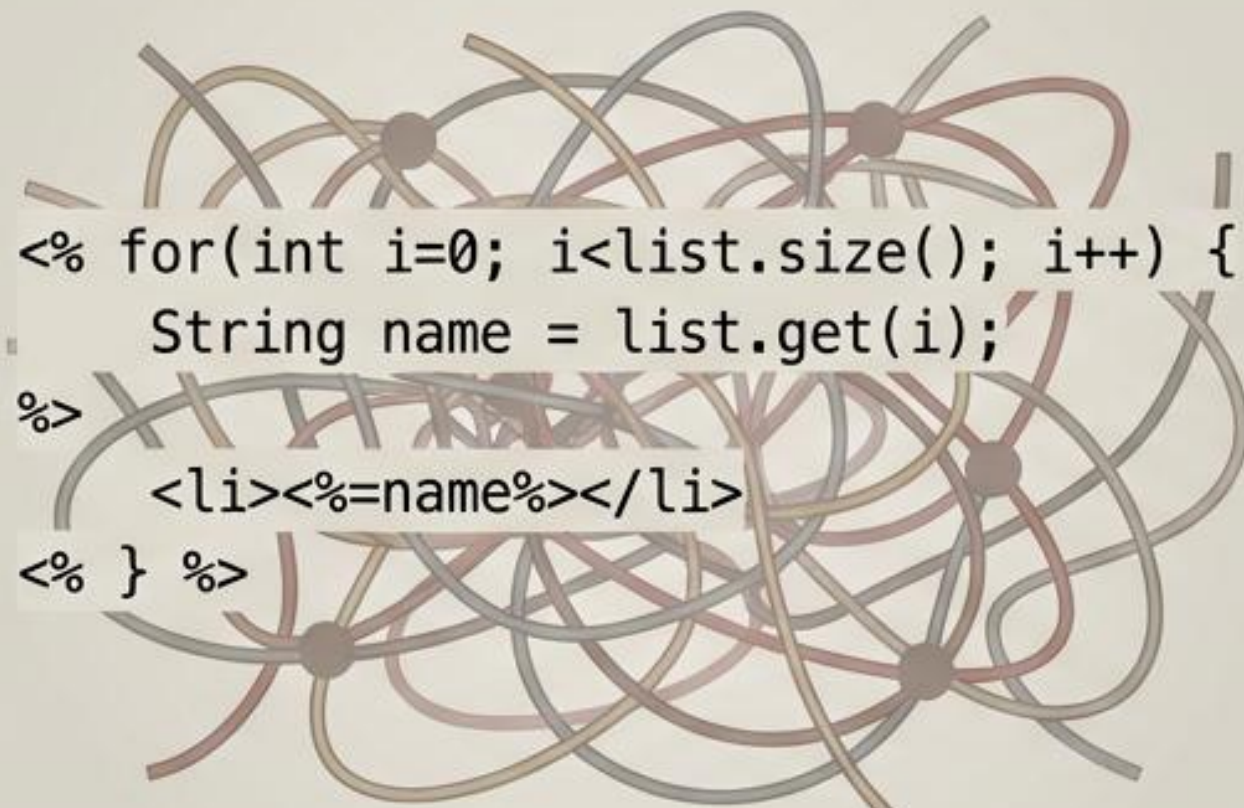


MVC 구조에서 각 구성 요소가 담당하는 기술적 역할 정의

계층 (Layer)	담당 기술	주요 역할
Controller	Servlet	요청 처리 및 비즈니스 로직 제어
View	JSP / EL / JSTL	데이터 출력 및 화면 로직 처리
Model	DAO / Oracle SQL	데이터베이스 접근 및 CRUD 처리

스파게티 코드와의 작별

과거의 방식



```
<% for(int i=0; i<list.size(); i++) {  
    String name = list.get(i);  
%>  
    <li><%=name%></li>  
<% } %>
```



모던 아키텍처

```
<c:forEach var="name" items="${list}">  
    <li>${name}</li>  
</c:forEach>
```

JSP 화면 내에 자바 로직(<% %>)이 섞이면 유지보수가 불가능해집니다.

목적: View(JSP) 계층에서 순수 Java 코드를 0%로 줄이는 것. EL과 JSTL은 이를 가능하게 하는 표준 도구입니다.

데이터의 여정 (MVC Architecture)

1. Model (Java/DAO)



데이터를 가져와
객체(VO, List)로 포장.

2. Controller (Java)



`request.setAttribute("키", 객체)`
또는 `session.setAttribute()`를 통해
데이터를 '수화물'로 등록

3. View (JSP/HTML)



EL과 JSTL이라는 '추출기'를
사용해 데이터를 꺼내어
화면에 렌더링



핵심 규칙: EL은 지역 자바 변수를 읽지 않습니다.
오직 Request, Session 등의 서버 저장소에 등록된 데이터만 읽을 수 있습니다.

데이터 브릿지: EL (Expression Language)

~~<%= ((StudentVO) request.getAttribute("vo")).getName() %>~~



`${vo.name}`

역할

데이터를 화면에 출력하는 표현식
(대체 `<%= %>`).

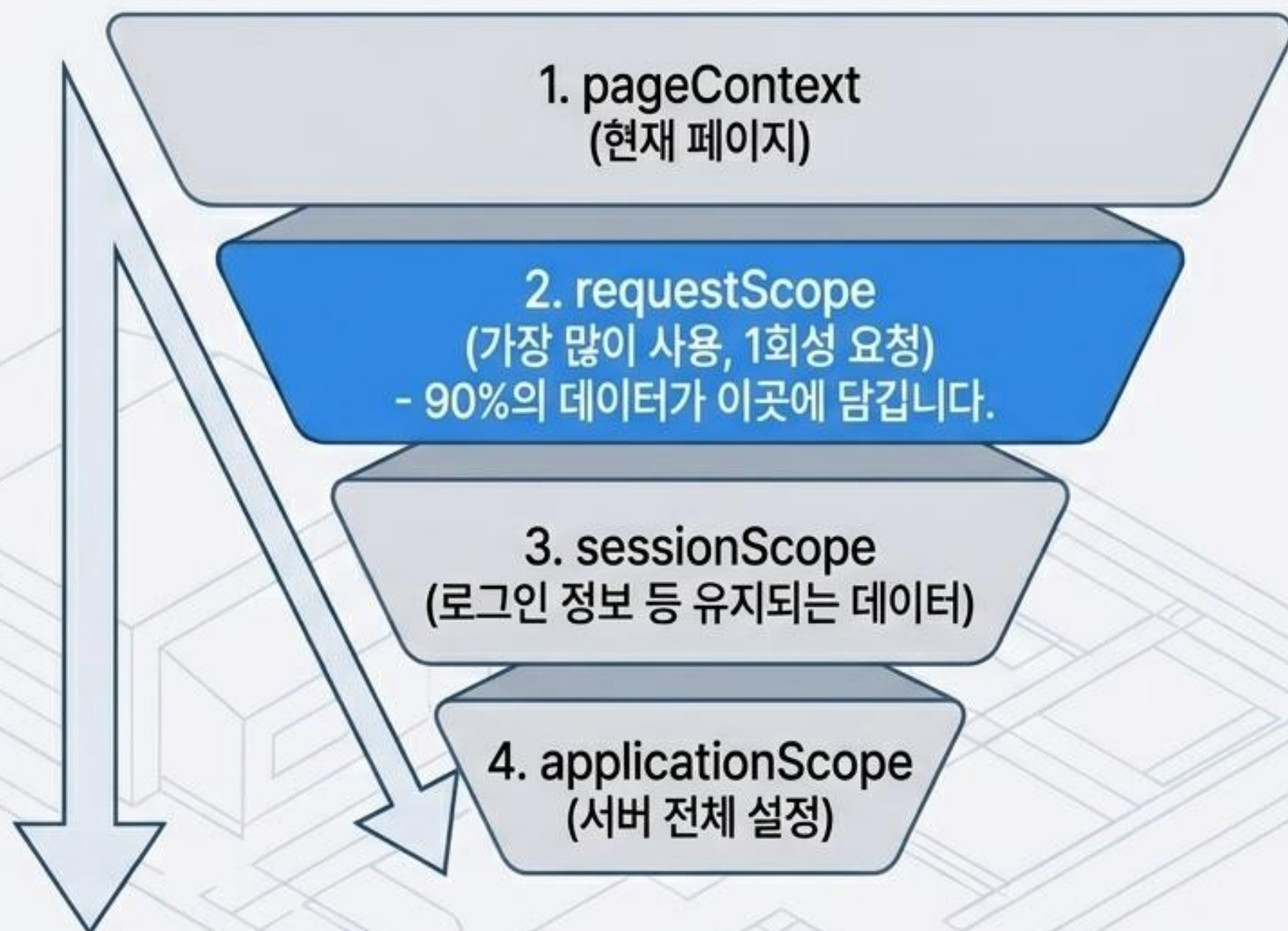
스마트한 속성 접근

`${vo.name}`은 내부적으로 자바의
`vo.getName()` 메서드를
자동 호출합니다.

Null 안전성

값이 null이어도 화면에 null을
출력하거나 에러를 뱉지 않고
공백(또는 0)으로 안전하게 처리합니다.

EL은 데이터를 어디서 찾아가? (Scope Funnel)



URL 파라미터 (param)

URL 파라미터(?id=admin)를 읽을 때는 `${param.id}` 사용.
(`getParameter("id")` 완벽 대체).

Key Takeaway

`${id}`라고만 적으면 위에서부터 아래로 검색합니다. 이름이 겹친다면 `${sessionScope.id}`처럼 명시적으로 지정하세요.

EL 연산자 매트릭스 (The Dictionary)

산술 (Math)

`+`, `-`, `*`, `/` (div), `%` (mod)

⚠ 정수/정수는 실수로 계산됨 ($5/2 = 2.5$)

⚠ `+`는 산술 전용. 문자열 결합은 `+=` 사용

논리 (Logical)

`&&` (and)

`||` (or)

`!` (not)

비교 (Relational)

`==` (eq)

`!=` (ne)

`<` (lt)

`>` (gt)

`<=` (le)

`>=` (ge)

유틸리티 (Utility)

`empty`: 객체가 `null`이거나 리스트가 비어있으면 `true` 반환 (`${empty param.name}`).

삼항 연산자: `${조건 ? 참 : 거짓}`.

로직 빌더: JSTL 생태계 (JSTL Ecosystem)

Core (c:)

제어문(조건/반복),
변수 선언, 화면 이동.

```
<%@ taglib prefix="c"  
uri="jakarta.tags.core" %>
```

Format (fmt:)

날짜 변환, 숫자(통화) 포매팅.

```
<%@ taglib prefix="fmt"  
uri="jakarta.tags.fmt" %>
```

Functions (fn:)

문자열 조작 (length,
substring, replace).

```
<%@ taglib prefix="fn"  
uri="jakarta.tags.functions" %>
```

XML 문법 기반이므로 여는 태그와 닫는 태그를 일치시켜야 하며, 속성값은 반드시 큰따옴표("")로 감싸야 합니다.

흐름 제어: 조건과 분기 (Branching)

단일 조건 - c:if

```
<c:if test="${i % 2 == 0}">  
    짝수  
</c:if>
```

⚠ JSTL에는 <c:else>가 존재하지 않습니다.
단일 조건만 처리 가능.

다중 조건 - c:choose

```
<c:choose>  
    <c:when test="${star == 5}">★★★★★</c:when>  
    <c:when test="${star == 4}">★★★★☆</c:when>  
    <c:otherwise>평가 없음</c:otherwise>  
</c:choose>
```

Java의 switch나 if-else if를 완벽히 대체합니다.

데이터 반복: The ForEach Engine



1. 전통적 반복 (for i=1; i<=10)

```
<c:forEach var="i" begin="1" end="10" step="1">
```

! step에 음수(-1)를 사용하여 감소하는 루프는 불가능합니다. 오직 증가식만 지원.

2. 컬렉션 순회 (for-each)

실무에서 90% 사용

```
<c:forEach var="vo" items="${list}">
```

리스트에서 객체를 하나씩 꺼내어 vo라는 이름의 지역 변수(var)로 화면에 던져줍니다.

varStatus의 숨겨진 힘 (배열 인덱싱)

```
<c:forEach var="name" items="${list}" varStatus="s">  
    ...  
</c:forEach>
```

s.index

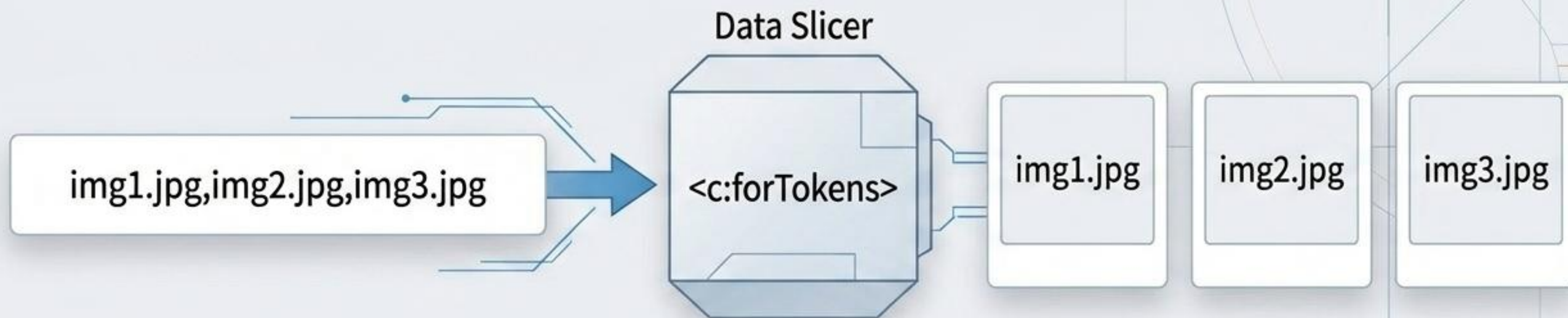
0부터 시작하는 실제 배열 인덱스.
(예: 다른 리스트와 병렬 매칭할 때 `${sexs[s.index]}`)

s.count

1부터 시작하는 반복 횟수.
(화면에 번호를 매길 때 1. 홍길동, 2. 박문수 → `${s.count}. ${name}`)

두 개의 서로 다른 리스트(이름 리스트, 성별 리스트)를 하나의 반복문에서 합쳐서 출력해야 할 때 s.index가 완벽한 해결책이 됩니다.

데이터 슬라이싱: ForTokens



```
<c:forTokens items="${colors}" delims=","  
  var="color">  
  <span class="badge">${color}</span>  
</c:forTokens>
```

- 자바의 StringTokenizer를 View 계층으로 가져온 태그.
- items: 자를 대상 문자열.
- delims: 구분자 (예: 쉼표 ,).
- var: 잘려진 각각의 토큰을 담을 변수명.

포매팅 레이어 (fmt: 숫자와 날짜)

대T스포츠

Wed Dec 11
10:00:00 KST
2023

날짜 - Date

```
<fmt:formatDate  
  value="${today}"  
  pattern="yyyy-MM-dd"/>
```

UI 보경

2023-12-11

숫자 - Currency

```
<fmt:formatNumber  
  value="${price}"  
  type="currency"/>
```

1234567

₩1,234,567

(자동으로 3자리 콤마 처리)

SimpleDateFormat과 DecimalFormat을 자바 컨트롤러에서 지워버리세요.
포매팅은 View(JSP)의 고유한 책임입니다.

문자열 조작 유틸리티 (fn: String Functions)

자바의 String 클래스 메서드를 EL `${ }` 내부에서 직접 호출할 수 있게 해주는 기능.

길이 확인	<code>\${fn:length(msg)}</code>	문자열의 길이나 리스트의 사이즈 반환.
문자열 자르기	<code>\${fn:substring(msg, 0, 3)}</code>	(예: "홍길동입니다" → "홍길동")
문자열 치환	<code>\${fn:replace(msg, '홍', '김')}</code>	(예: "홍길동" → "김길동")

Tip : 주로 모델에서 데이터를 받아왔지만 화면 UI의 제약으로 인해 문자열을 살짝 다듬어야 할 때 유용합니다.

실전 뷰 레이어 블루프린트 (Synthesis)

```
<c:forEach var="vo" items="${list}">  
  <div class="thumbnail">  
      
    <p class="caption">${vo.goods_name}</p>  
  </div>  
</c:forEach>
```

Controller에서
넘겨준 상품 리스트.

Getter 없이 속성명으로 직접 접근하는 EL의 우아함.

자바 코드(<% %>)가 단 한 줄도 없는,
디자이너와 퍼블리셔가 사랑하는 완벽한 클린 코드.

EL & JSTL 마스터 치트시트

EL 연산자 & Scope

eq (==), ne (!=),
div (/), mod (%)

empty (Null/빈 객체 확인)

`${param.id}` (요청 파라미터)

JSTL Core 태그

```
<c:set var="x" value="1"/>
```

```
<c:if test="${조건}">
```

```
<c:choose> / <c:when> /  
<c:otherwise>
```

```
<c:forEach var="item"  
  items="${list}">
```

Format & Functions

```
<fmt:formatDate  
  value="${d}"  
  pattern="yyyy-MM-dd"/>
```

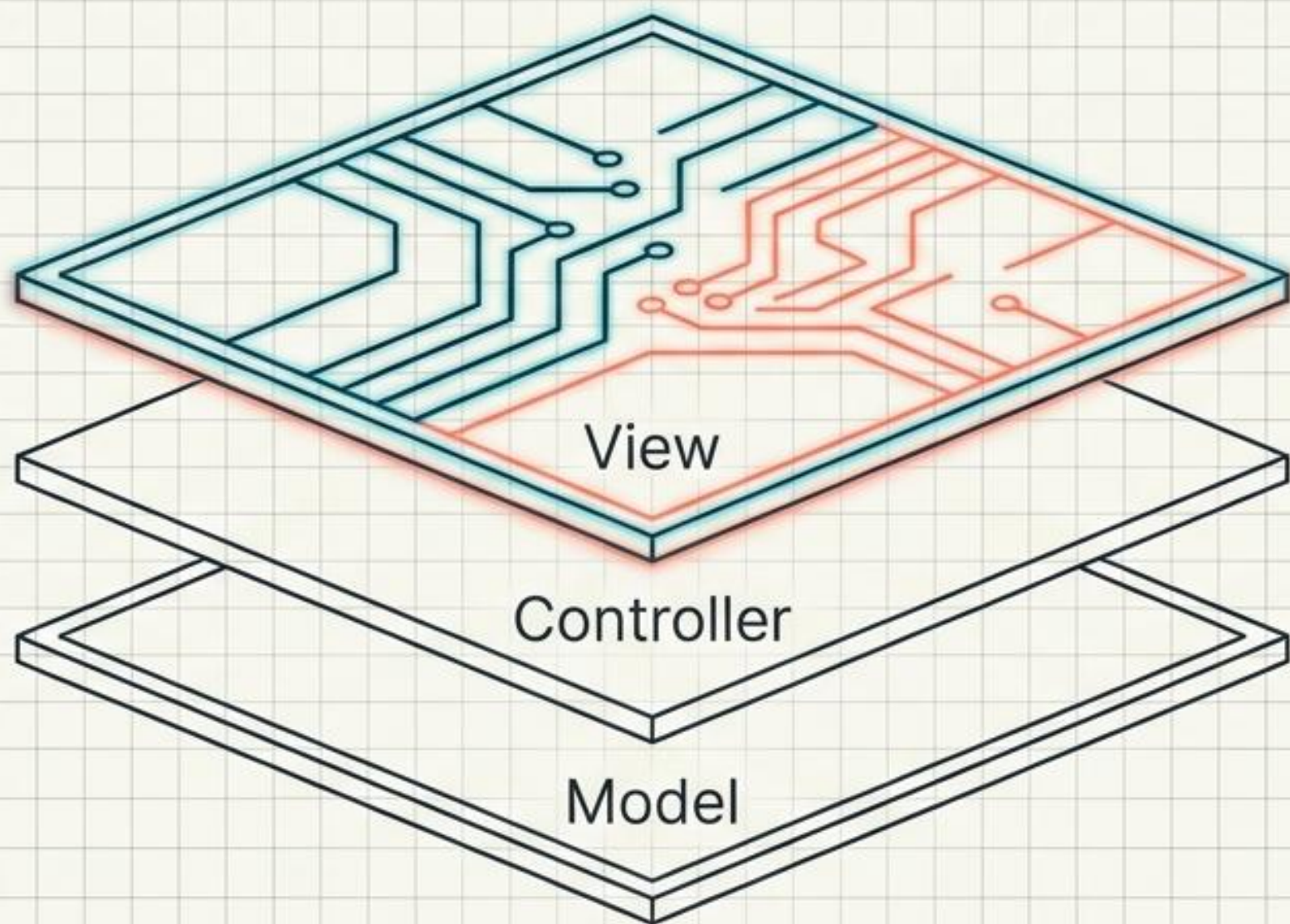
```
<fmt:formatNumber  
  value="${n}"  
  type="currency"/>
```

```
${fn:length(text)}
```

Clean Code in the View Layer.

Scriptless JSP: 모던 View 아키텍처

EL과 JSTL을 활용한 완벽한 UI 코드 설계 가이드



왜 EL과 JSTL인가?



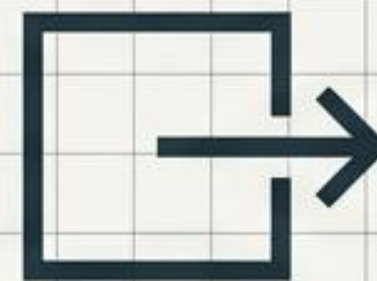
자바 코드의 완전한 배제 (Scriptless)

- JSP 내 `<% %>` (스크립틀릿) 제거.
- HTML 태그와 유사한 구조로 이질감 감소 및 보안성 향상.



명확한 역할 분담과 협업

- 복잡한 로직이 없는 코드로 웹 디자이너 및 퍼블리셔와의 협업 용이.



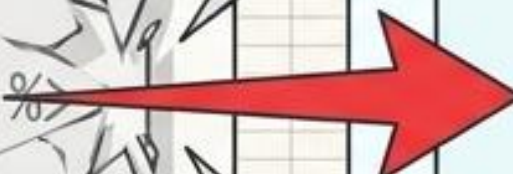
View 계층의 순수성 유지

- JSP는 비즈니스 로직을 처리하지 않음.
- 전달받은 결과값만 출력하는 전송용 파일로 역할 제한.

UI 코드의 진화: 스크립틀릿의 종말

Before: 혼돈의 자바 코드

```
<%= request.getAttribute("name") %>
```



길고 복잡하며 HTML 구조를 파괴함.

After: 우아한 데이터 접근

```
${name}
```

간결하며 순수 HTML 텍스트처럼 자연스럽게 융화됨.

자바 코드를 최소화하는 정책의 핵심은 가독성의 극대화입니다.

JSTL 환경 설정: 3대 필수 라이브러리

JSP 상단 taglib 지시자 선언. 환경(버전)에 따라 jakarta.tags 또는 java.sun.com URI를 선택합니다.

Core (코어)

```
prefix="c"
```

기능: 제어문(조건/반복), 변수 선언, URL 처리

Formatting (포매팅)

```
prefix="fmt"
```

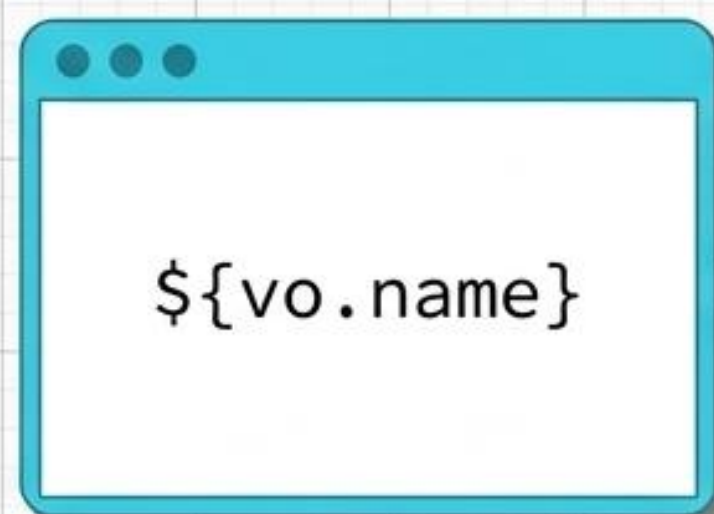
기능: 날짜 및 숫자(천 단위 콤마 등) 서식 변환

Functions (함수)

```
prefix="fn"
```

기능: 문자열 길이 계산 및 자르기 기능 지원

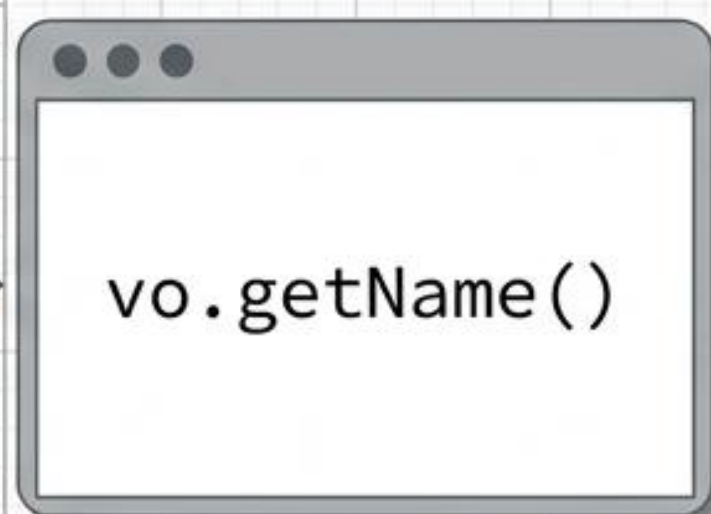
EL 엔진의 비밀: 자동 Getter 매핑



마침표 연산자 사용



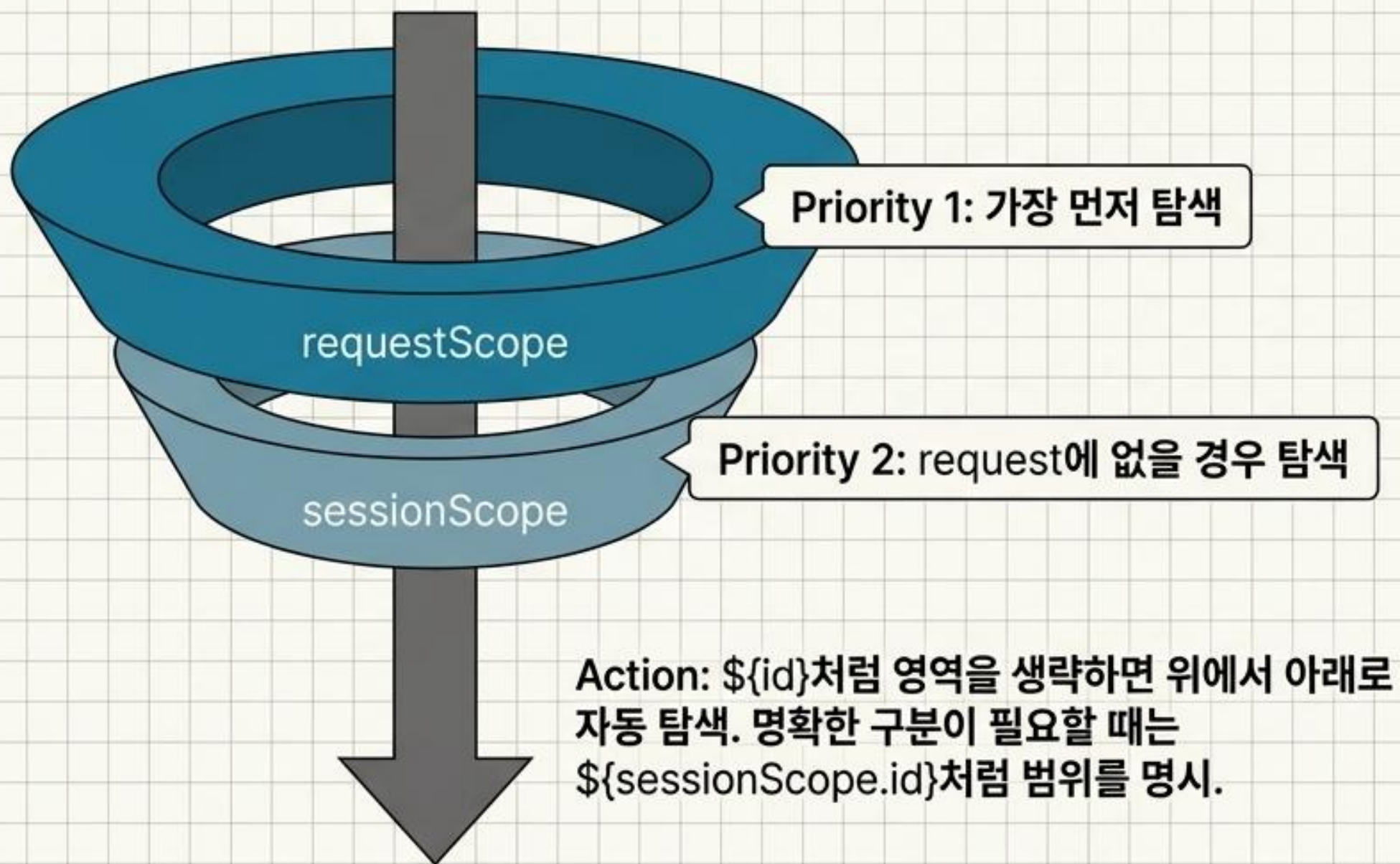
EL 엔진 (자동 매핑)
자바 빈즈(VO) 규약 탐색



내부적으로
public Getter 자동 호출

개발자가 복잡한 자바 메서드를 작성할 필요 없이, 객체의 이름과 속성명만으로 데이터에 접근합니다.
단, VO 클래스는 반드시 private 변수와 public Getter를 가져야 합니다.

데이터 탐색 계층: EL의 Scope 우선순위



기타 주요 내장 객체

param: 외부 전달 데이터.
request.getParameter()와 동일.
(예: \${param.name})

pageContext: 현재 실행 중인 JSP
페이지의 시스템/환경 정보 접근.

EL 연산자의 고유한 문법

Null의 안전한 처리

산술 연산 시 null을 자동으로 0으로 인식.

```
${null + 7}
```

➔ 결과: 7

문자열 결합의 절대 규칙

+는 오직 산술 연산 전용. 문자열 결합은 반드시 += 사용.

```
${"5\" + 7} ➔ 12 (정수 변환)
```

```
${"A" += 7} ➔ A7
```

산술 & 논리 키워드

기호 대신 영문 키워드 사용 가능.
나눗셈 시 정수끼리 나뉘도 결과는 실수.

/ ➔ div
% ➔ mod
&& ➔ and
|| ➔ or

empty 연산자

값이 null이거나, 비어있거나, 컬렉션의 데이터가 없을 때 true 반환.

```
${empty param.name}
```

(요청 파라미터 유무 판별 시 활용)

데이터의 처리: Memory vs Display



<c:set> (저장소)

- 핵심 역할: JSP 내에서 사용할 변수 선언 및 저장
- 자바 대응: request.setAttribute()

```
<c:set var="today" value="2023-12-11" />
```

비유: 나중에 사용하기 위해
바구니에 담아두기



<c:out> (화면)

- 핵심 역할: 브라우저 화면에 데이터를 직접 출력
- 자바 대응: out.println(), <%= %>

```
<c:out value="${name}" />
```

비유: 바구니에 든 내용을
사람들에게 보여주기

논리 제어 매트릭스: 단일 vs 다중 분기

기능 구분	<c:if>	<c:choose>
비교 대상	자바의 if	자바의 switch, if-else if
조건 개수	단일 조건 처리	다중 조건 처리 (<c:when>)
Else 처리	✗ 불가 (지원하지 않음)	✓ <c:otherwise>를 통해 완벽 지원
주 용도	간단한 참/거짓 확인 시	여러 분기가 있거나 기본값 설정이 필수일 때

Else 없는 <c:if>, 어떻게 해결할 것인가?

조건 분기 설계

부정 연산자 활용
(Workaround)

! 또는 not 연산자를 활용해
정반대 조건의 <c:if>를 추가 작성.

```
<c:if test="${sessionScope.id == null}">
    로그인 필요
</c:if>
<c:if test="${!(sessionScope.id == null)}">
    환영합니다
</c:if>
```

✓ c:choose의 우아한 구조
(Best Practice)

otherwise를 통한 깔끔한
기본값(Else) 처리.

```
<c:choose>
  <c:when test="${sessionScope.id == null}">
    로그인 필요</c:when>
  <c:otherwise>환영합니다</c:otherwise>
</c:choose>
```


반복 루프의 해부학: <c:forEach>

```
<c:forEach var="name" items="${list}" varStatus="s">
```

index (0-based)

배열/컬렉션의 실제 인덱스 (0부터 시작).

용도: 다른 리스트의 동일 위치 데이터 참조.
(예: `${sList[s.index]}`)

count 역할 (1-based 대체 방식)

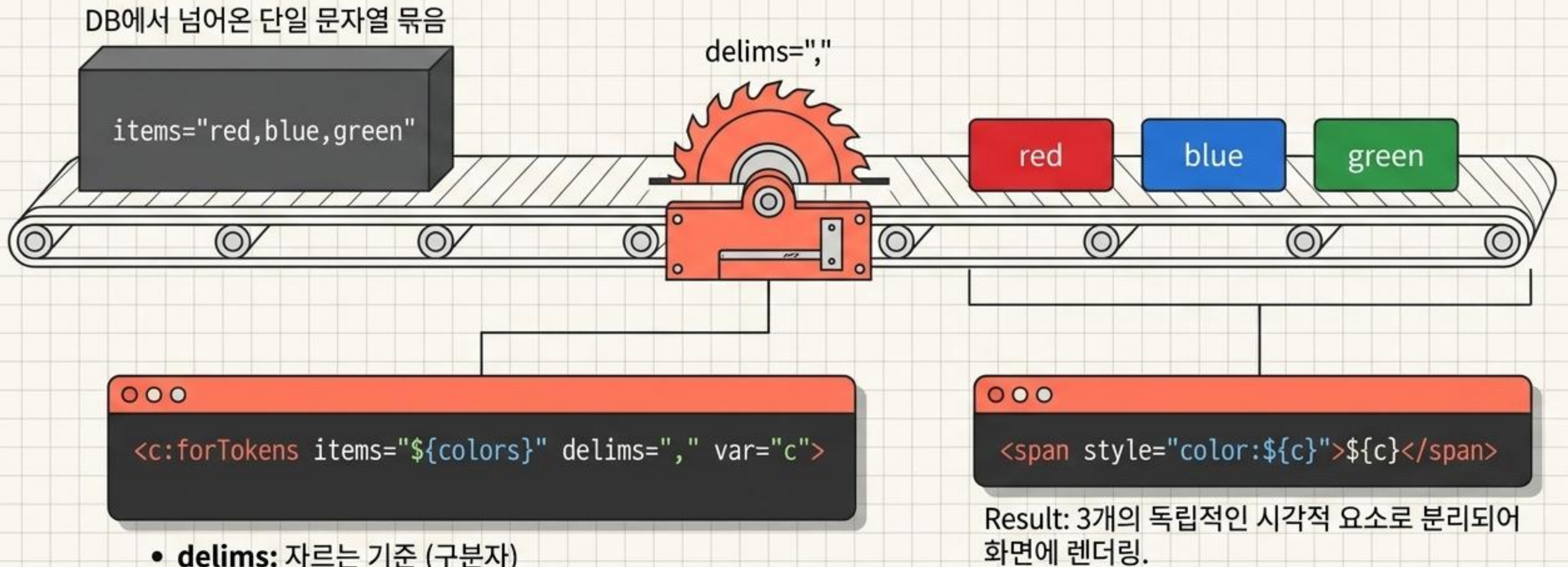
화면에 순번(번호)을 출력하기 위해 사용.

구현: `${s.index + 1}`을 사용하여 1부터 시작하는 순번 완성.



구조적 한계: 감소식(Decrement) 사용 불가. step은 무조건 양수(증가식)여야 합니다.
역순 출력이 필요하다면 모델(Java)에서 정렬 후 전달해야 합니다.

문자열 즉석 파싱: <c:forTokens>



- **delims:** 자르는 기준 (구분자)
- **var:** 분리된 각 토큰을 담는 임시 변수

View 계층에서의 깔끔한 라우팅

기능은 화면 이동(URL 이동)으로 완전히 동일하지만, 구현 방식과 아키텍처적 목적이 다릅니다.



(과거의 방식: Scriptlet)

```
<% response.sendRedirect("파일명"); %>
```

- 문제점: View 계층에 자바 로직 노출.

(모던 View 아키텍처: JSTL)

```
<c:redirect url="파일명"/>
```

- 장점: 태그 기반 설계로 자바 코드를 최소화하고, 순수 View로서의 구조적 완전성 유지.

데이터의 시각적 포매팅 (<fmt>)

DB의 가공되지 않은 데이터를 사용자가 읽기 편한 형태로 출력.



커스텀 제어 (pattern 속성)

개발자가 원하는 서식을
직접 세밀하게 지정.

```
<fmt:formatNumber value="{price}"  
  pattern="###,###" />
```

결과: 30000 ➡ 30,000



사전 정의된 규격 (type 속성)

정해진 로케일 규칙에 따라
통화 기호 등을 자동으로 적용.

```
<fmt:formatNumber value="{1234567}"  
  type="currency" />
```

결과: 화폐 단위 기호 자동 포함 (예: ₩1,234,567)

날짜 역시 <fmt:formatDate value="{today}" pattern="yyyy-MM-dd"/> 형태로 제어 가능.

완성된 설계도: 스크립트 없는 JSP의 조화

1. 선언 - 필수
라이브러리 엔진 가동.

```
<%@ taglib prefix="c"... %>  
<%@ taglib prefix="fmt"... %>
```

2. 제어 - empty
연산자와 JSTL 다중
분기를 통한 상태 체크.

```
<c:choose>  
  <c:when test="${empty sessionScope.user}">  
    <!-- Login required UI -->  
  </c:when>
```

3. 반복 - cart 객체 순회
및 상태 추적.

```
<c:otherwise>  
  <c:forEach var="item" items="${cart}" varStatus="s">  
    <span>${s.index + 1}. ${item.name}</span>  
    <fmt:formatNumber value="${item.price}" pattern="###,###"/>  
  </c:forEach>  
</c:otherwise>  
</c:choose>
```

4. 데이터 추출 - EL
자동 Getter 매핑 및
순번(1-based) 출력.

5. 포매팅 - UI 디자인
요구사항에 맞춘 데이터
시각화.

자바 코드는 완벽히 사라졌습니다. 오직 설계만 남았습니다.

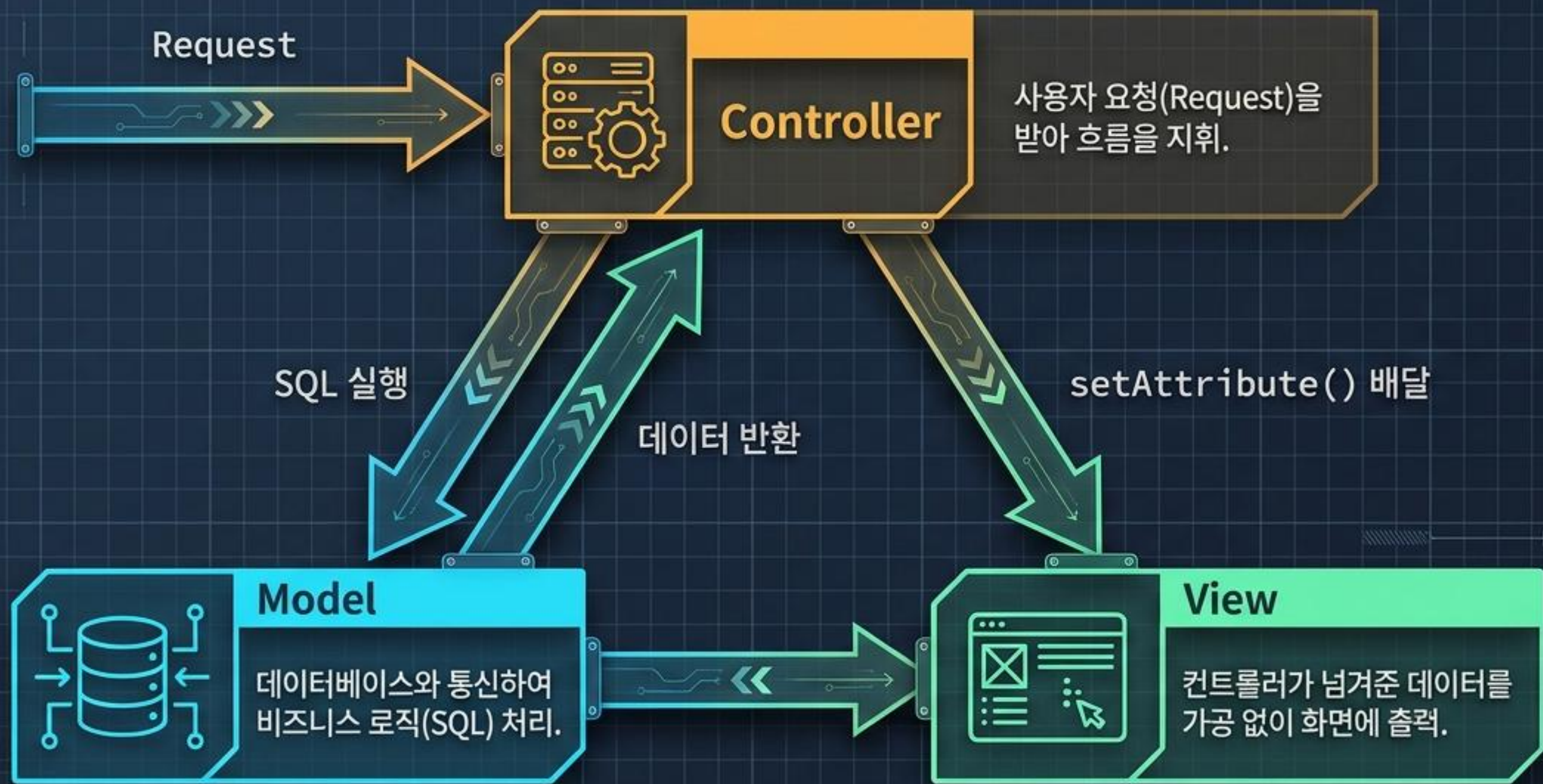
JSP MVC & Oracle: 아키텍처 블루프린트



데이터의 흐름으로 완성하는 웹 개발 마스터 가이드

SQL, JDBC, Servlet, JSP, EL/JSTL 통합 설계도

1 Request = 1 Pipeline (MVC 3계층 아키텍처)



데이터 창고의 4대 핵심 명령어 (CRUD)



SELECT (조회)

```
SELECT * FROM member  
WHERE id='hong' AND pwd='1234';
```

(실무: 로그인 및 목록 조회)



INSERT (추가)

```
INSERT INTO member  
VALUES (member_seq.nextval, 'hong', ...);
```

(실무: 회원가입, SEQUENCE로 자동 번호 부여)



UPDATE (수정)

```
UPDATE member SET age=30  
WHERE id='hong';
```

(실무: 회원정보 수정)



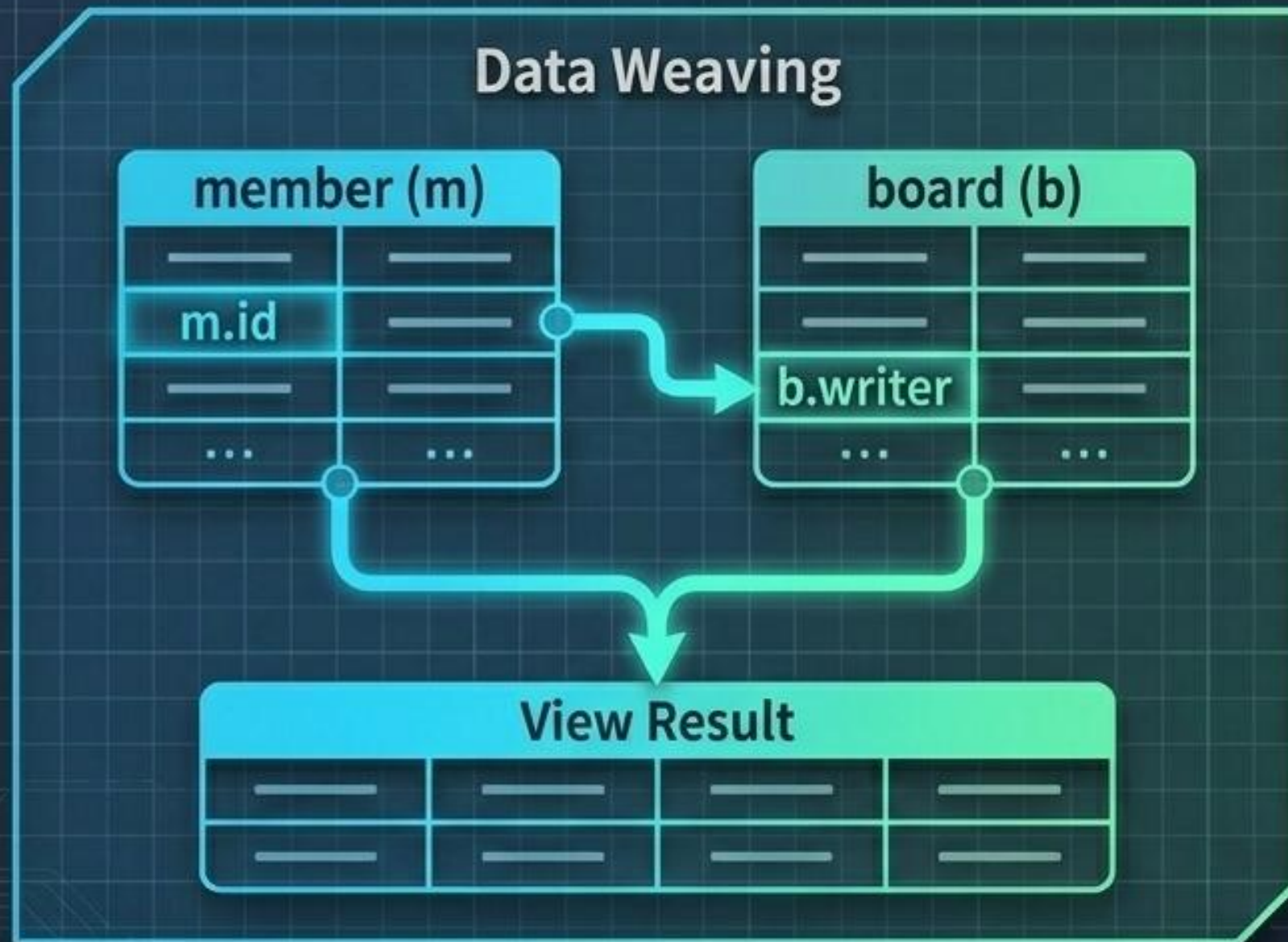
DELETE (삭제)

```
DELETE FROM member  
WHERE no=1;
```

(실무: 게시물 삭제)

데이터의 결합: INNER JOIN

분리된 회원 데이터와 게시판 데이터를 하나로 묶어 화면에 전달하는 핵심 기술.

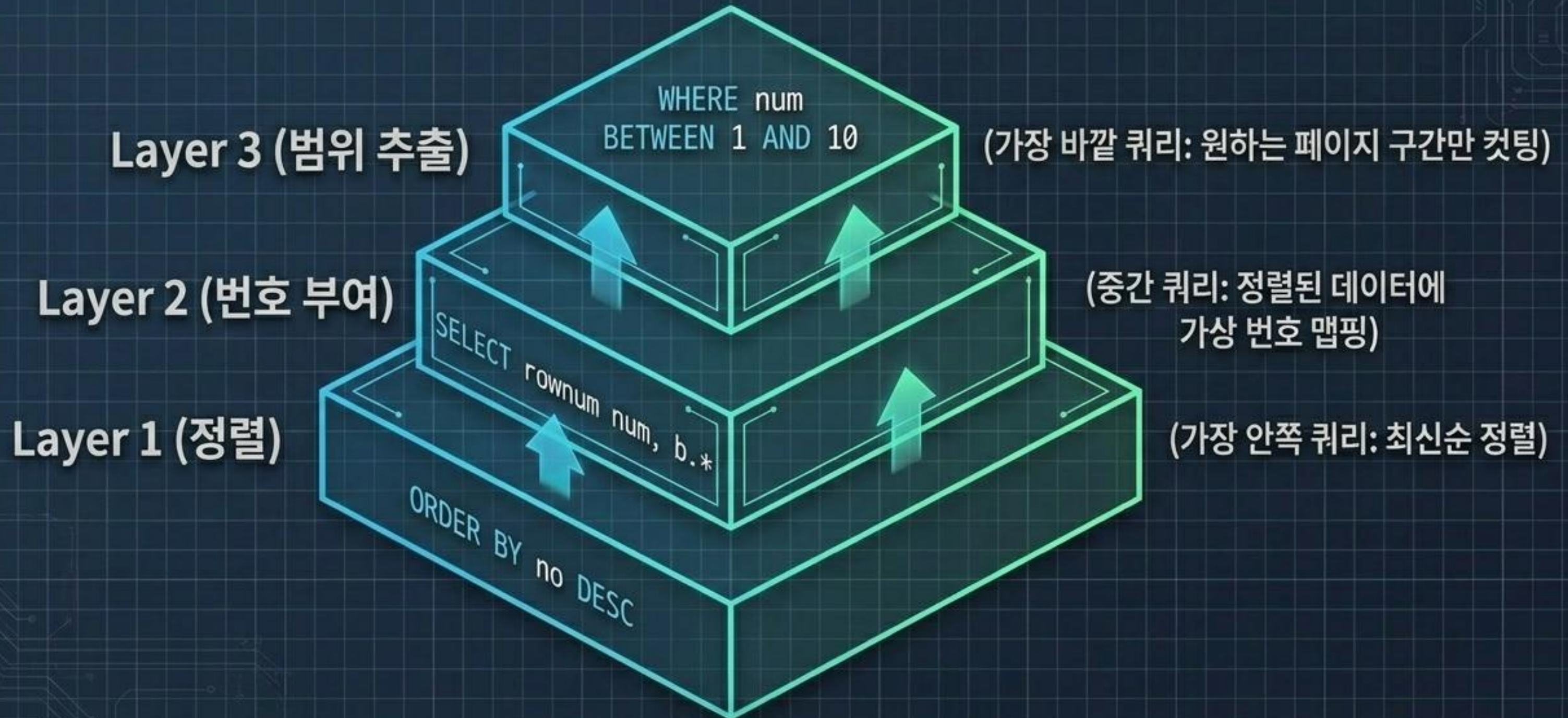


```
SELECT m.name, b.title  
FROM member m  
JOIN board b ON m.id = b.writer;
```

실무 게시판 및 댓글 구현 시 100% 사용되는 필수 SQL.

서브쿼리 해부: 게시판 페이징 (The Paging Layer)

오라클 실무에서 가장 복잡하고 중요한 3중 감싸기 구조.



데이터 운반책: JDBC와 DAO 패턴



DAO

(Data Access Object)

SQL 처리만을
전담하는 클래스.

PreparedStatement

```
String sql = 'SELECT * FROM member WHERE id=?';
```

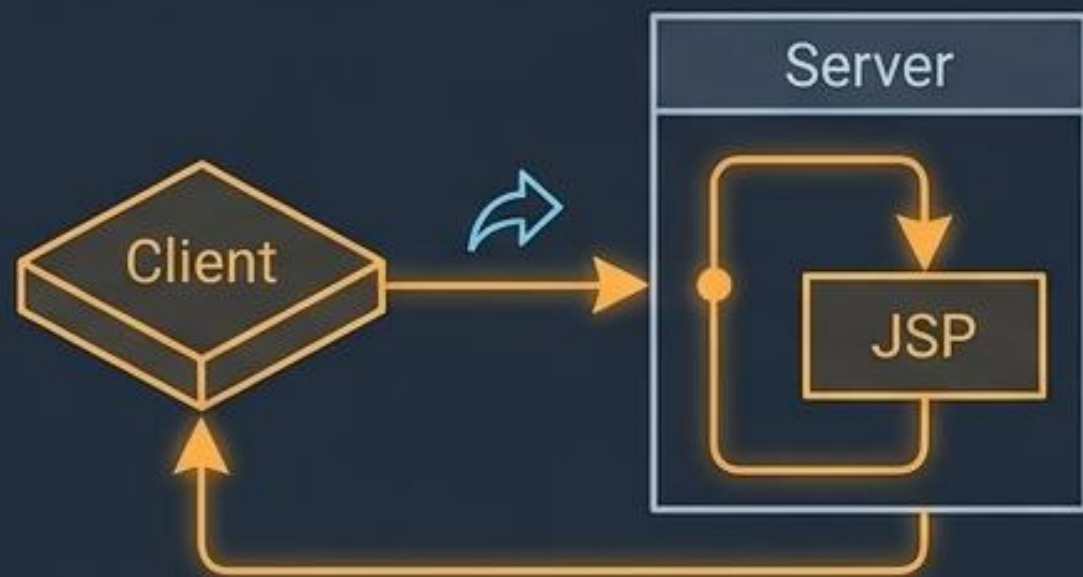
?를 사용하여 동적 데이터 바인딩.
SQL Injection 방어 핵심.

ResultSet

쿼리 결과를
Java 객체(List<Member>)로
변환하여 Controller로 전달.

지휘관의 선택: 페이지 이동 방식 (Forward vs Redirect)

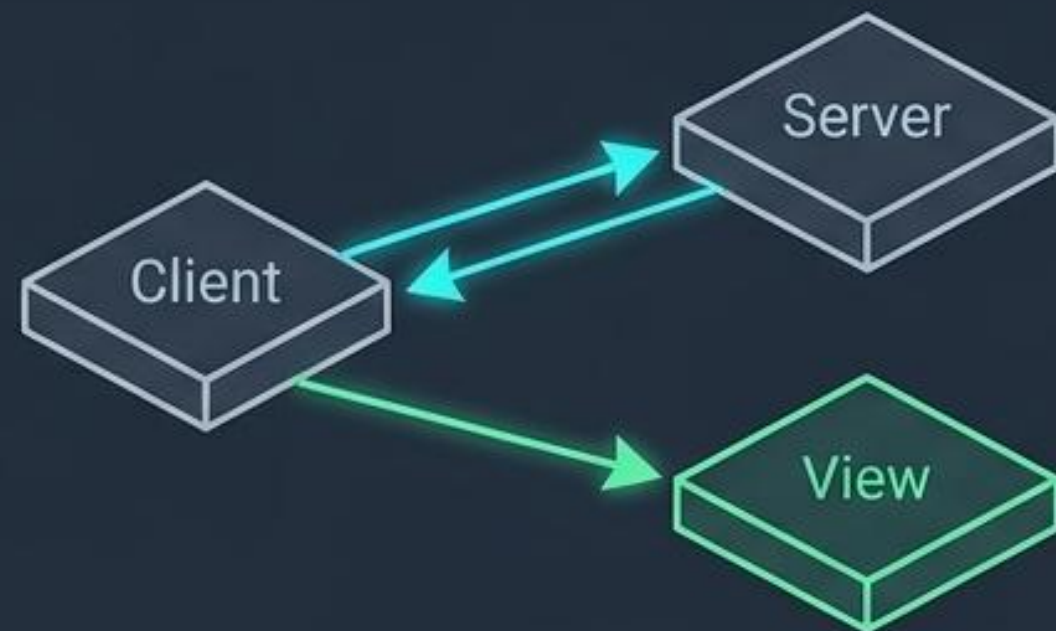
⚡ Forward (포워드)



```
rd.forward(request, response);
```

- 서버 내부 이동 (주소창 변경 없음).
- request 객체 데이터 유지 (데이터 전달 시 필수).
- 속도: 빠름.

🐢 Redirect (리다이렉트)



```
response.sendRedirect('url');
```

- 브라우저에게 재요청 지시 (주소창 변경됨).
- request 데이터 초기화 (완전한 새 요청).
- 속도: 느림.

상태 유지의 무기: Cookie vs Session



Cookie (쿠키)

- 저장소: 클라이언트 (브라우저)
- 보안: 낮음 (조작 가능)

```
Cookie cookie = new Cookie('id', 'hong');  
response.addCookie(cookie);
```



Session (세션)

- 저장소: 서버 (안전한 메모리)
- 보안: 높음 (로그인 유지의 핵심)

```
session.setAttribute('id', 'hong');
```


동적 뷰의 진화: 스크립틀릿에서 EL/JSTL로

Before

과거의 날것 (스크립틀릿)

Java와 HTML이 엉켜 유지보수 최악

```
<% String name =  
(String)request.getAttribute('name');  
out.println(name);  
%>  
  
<% for(Member m : list) { %>
```



After

현재의 모듈화 (EL/JSTL)

화면에서 Java 코드를 제거

```
${name}  
  
<c:forEach var='m' items='${list}'>
```

Rule: JSP 화면에서는 Java 코드를 0%로 만드는 것이 실무의 핵심 목표.

EL (Expression Language)의 데이터 탐색 생존주기



기본 문법

`${name}`

내부적으로 Getter 자동 호출

파라미터 즉시 접근

`${param.id}`

`request.getParameter('id')` 와 완벽히 동일.

JSTL Core: 화면의 흐름을 제어하는 논리 블록

변수/출력 (XSS 방지)

```
<c:set var='name' value='홍길동' />  
<c:out value='${name}' />
```

단일 조건

```
<c:if test='${age} >= 20'>  
간단한 true/false 분기
```

다중 조건 (Switch)

```
<c:choose>  
  <c:when>  
  <c:otherwise>  
</c:choose>
```

반복 (The MVP)

```
<c:forEach var='m' items='${list}' varStatus='s'>  
MVC에서 List<Member>를 화면에 뿌릴 때 100% 사용됨.
```


JSTL 확장팩: 데이터 포매팅과 문자열 가공



FMT (데이터 변환기)



숫자/통화

```
<fmt:formatNumber value='10000'  
  type='currency' />
```

결과: ₩10,000

날짜

```
<fmt:formatDate value='${today}'  
  pattern='yyyy-MM-dd' />
```

FN (문자열 유틸리티)



길이 측정

```
${fn:length(list)}
```

포함 여부

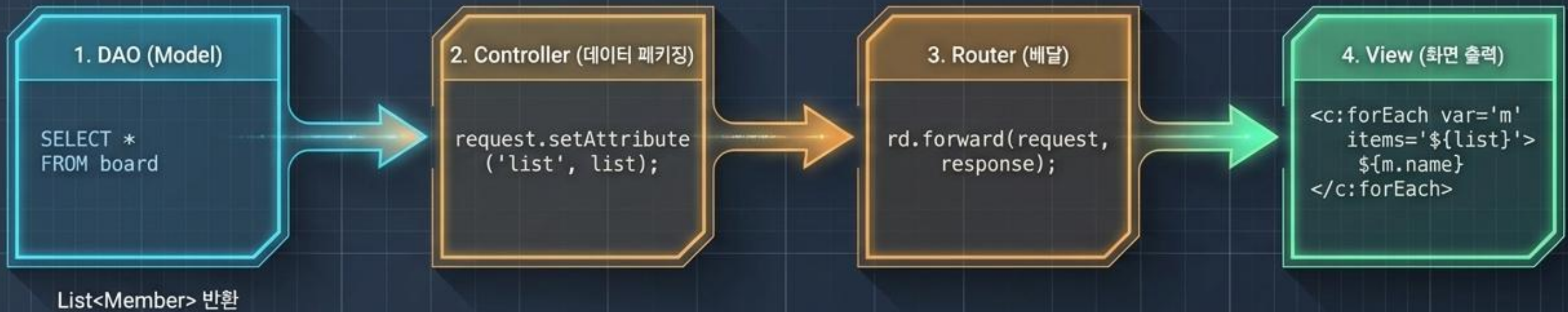
```
${fn:contains(str, 'java')}
```

치환 (Replace)

```
${fn:replace(str, 'java', 'jsp')}
```


궁극의 실무 조합 (1 Request = 1 Pipeline)

이것이 게시판, 쇼핑몰을 만드는 완벽한 표준 패턴입니다.



MVC 아키텍처 역할 분담 명세서

Servlet (Controller)

흐름 지휘 / 비즈니스 로직 / 요청과 응답 처리.

DAO (Model)

Oracle 통신 / SQL 실행 / 데이터 객체화.

JSP (View)

HTML 렌더링 / 화면 UI 레이아웃.

EL (View-Data)

request / session 객체의 데이터 추출 및 출력.

JSTL (View-Logic)

HTML 내에서의 안전한 반복문 및 조건문 처리.

웹 개발의 진화와 당신의 다음 단계

당신의 다음 단계

